gr00t-arm

A desk robot arm that learns from demonstrations with NVIDIA GR00T

How it connects to GR00T, and how to build, wire, calibrate, train and run it

Yishane Kao

October 2026

Contents

I How the arm connects to GR00T **3**

1 The idea in one picture **3**

2 What GR00T is, in plain words **3**

3 Every GR00T concept, and where it lives on this arm **4**

4 The whole pipeline **4**

5 Presenting it in a humanoids talk **5**
5.1 A 3-minute script 5
5.2 Live demo plan 5
5.3 Questions you might get 5

II Build and run instructions **6**

6 What you need **6**

7 Printing **6**
7.1 Fit test first 7

8 Laptop software setup **7**

9 Wiring **8**
9.1 Upload the firmware 8

10 Assembly **9**

11 Calibration **10**

12 First moves **10**

13 Collecting demonstrations **10**

14 Training ACT on the laptop (free, optional but recommended) **11**

15 Fine-tuning and running GR00T in the cloud **11**
15.1 One-time accounts (you do these) 11
15.2 Steps 11

16 Troubleshooting **12**

17 Where everything is **12**

Part I
How the arm connects to GR00T

1 The idea in one picture

GR00T is NVIDIA's *robot foundation model*: one neural network, pretrained on a huge amount of robot and human data, that turns **what a robot sees**, **what it is told** and **where its joints are** into **the next motions**. It was built for humanoids, but it is designed so a *new* body can be taught to it with a small amount of data. This arm is that new body: five $2 servos, a phone for eyes, and about 60 demonstrations.

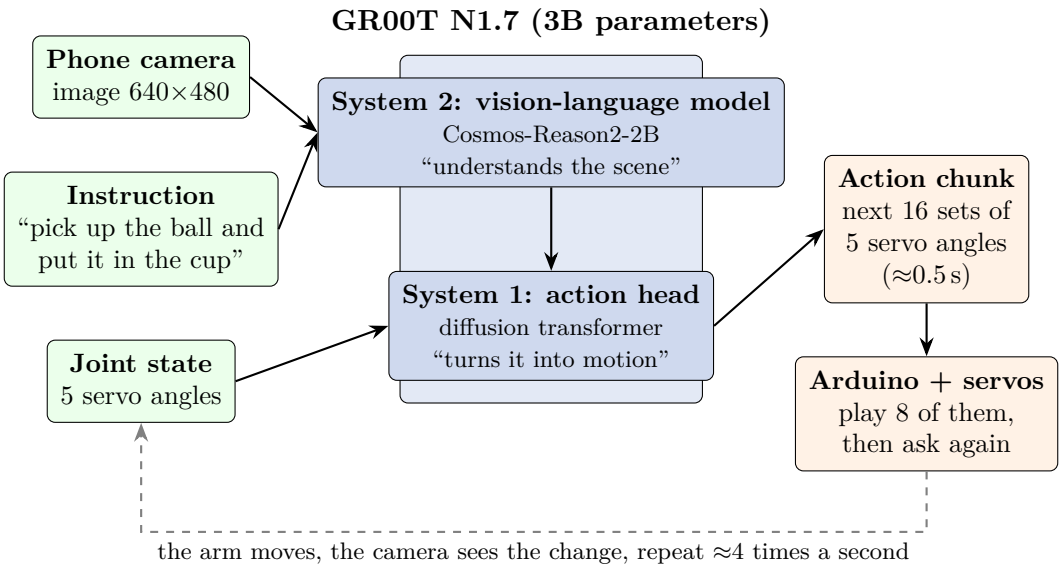


Figure 1: The control loop. Everything on the left is produced by this arm; the middle is GR00T; the right is sent back to the arm.

2 What GR00T is, in plain words

A foundation model for robots. Language models are pretrained on the internet and then adapted to a task. GR00T does the same for robot *motion*: it is pretrained on a mix of real robot recordings, simulated robots and videos of humans, then *post-trained* (fine-tuned) on a few dozen demonstrations of a specific robot doing a specific task [1, 2].

Two systems, like fast and slow thinking. The **System 2** part is a vision-language model: it looks at the camera image and reads the instruction, and produces a rich description of the situation. The **System 1** part is an *action head*, a diffusion transformer: it takes that description plus the robot's current joint angles and produces motor commands. System 2 is the "what is going on and what should happen", System 1 is the "how do my joints get there".

It outputs a short plan, not one move. The action head predicts an **action chunk**: the next 16 commands (about half a second). Predicting a chunk makes motions smooth and lets the model commit to a plan instead of dithering every frame. We play the first 8 and then ask again with a fresh image, so the arm keeps correcting itself.

How it generates actions: denoising. The action head starts from random numbers and refines them over a few steps (4 by default) into a sensible chunk, guided by what System 2 saw. This "flow matching" / diffusion trick is the same family of method image generators use, applied to joint angles instead of pixels.

One brain, many bodies. The big shared network is the same for every robot. What is specific to a body is small: how its joint state is read in and how actions are read out. GR00T has a reserved slot for robots it has never seen, `NEW_EMBODIMENT`. Our arm is registered there with 4 arm joints, 1 gripper and 1 camera.

3 Every GR00T concept, and where it lives on this arm

Table 1: Mapping GR00T's pieces onto the arm. File names are in the GitHub repo.

GR00T concept	On this arm	Where
Vision input	One phone camera on a fixed mast looking down at the work area (key `front`). Fixed so every demo looks the same.	mast + cradle STLs, `arm_io.py`
Language instruction	"pick up the ball and put it in the cup" stored with every demo	`recorder.py`
Proprioceptive state	5 servo angles in degrees: base, shoulder, elbow, wrist (`single_arm`) and gripper (`gripper`)	`export_dataset.py`
Embodiment	`NEW_EMBODIMENT` with our modality config	`gr00t_arm_config.py`
Dataset format	LeRobot v2.1 + `meta/modality.json` (parquet tables + mp4 videos)	`export_dataset.py`
Action chunk	16 future commands; we execute 8 then re-query	`run_policy.py`
Relative vs absolute actions	Arm joints are predicted as *changes* from the current angles (generalises better); the gripper as an absolute open/close target	`gr00t_arm_config.py`
Post-training	Fine-tune on ≈60 demos on a rented cloud GPU, ≈3000 steps	`gr00t/finetune.sh`
Inference	Model runs on the cloud GPU as a server; the laptop sends image + state, gets actions back over ZeroMQ	`gr00t/serve.sh`, `policy_client.py`
Demonstrations	Scripted pick-and-place (auto-collected) or keyboard teleop	`autodemo.py`, `teleop.py`

4 The whole pipeline

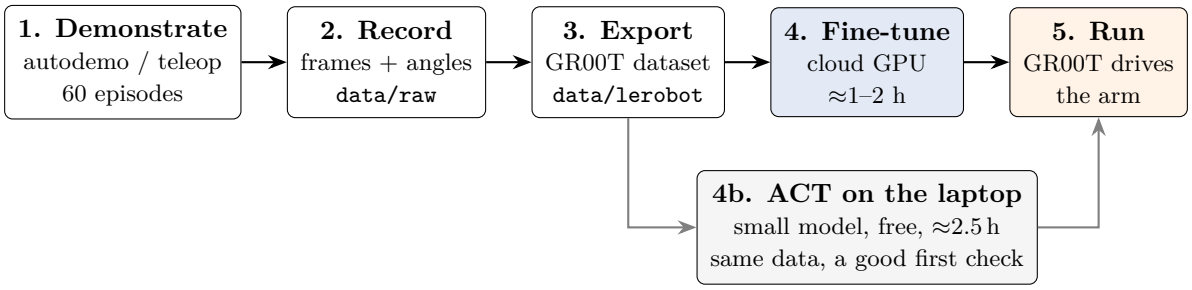


Figure 2: From demonstrations to a running policy. ACT [3] is a much smaller imitation-learning model that trains on the laptop; it checks the data before paying for a GPU.

5 Presenting it in a humanoids talk

5.1 A 3-minute script

What to say

1. **The problem.** "A humanoid has to turn what it sees and what it's told into motor commands for dozens of joints. Writing that by hand doesn't scale. NVIDIA's answer is GR00T, a foundation model for robots."
2. **Point at the arm.** "This is a $10 arm: five hobby servos and my phone as the camera. GR00T doesn't care that it's small. The recipe is the same one used for humanoids."
3. **Inputs and outputs** (Fig. 1). "Three inputs: the camera image, an instruction in English, and the current joint angles. One output: a chunk of the next 16 joint commands, about half a second of motion."
4. **Two systems.** "Inside, a vision-language model figures out *what* to do, and a diffusion transformer turns that into *how* the joints move. Slow thinking and fast moving."
5. **New body.** "GR00T has never seen this arm. I registered it as a new embodiment: 4 joints, a gripper, one camera, and fine-tuned it on about 60 demonstrations. That took an hour or two on a rented GPU."
6. **Demo** (Section 5.2).
7. **Back to humanoids.** "A humanoid is the same loop with more joints, more cameras and much more data. Pretraining on many bodies is what lets each new body learn from only a handful of demonstrations."

5.2 Live demo plan

Pick the most ambitious option you have working, and keep the next one ready.

1. **GR00T live.** Start `serve.sh` on the cloud GPU before class, connect with `run_policy.py gr00t`. Needs internet. Put the ball on a marked spot, run it.
2. **ACT live (no internet).** Same pipeline, smaller model running on the laptop: `run_policy.py act`. Say clearly that this is the small model trained on the same data.
3. **Scripted demo.** `autodemo.py -spot B2` always works. Explain that this script is how the training demonstrations were collected, and that the learned policy never sees the spot name, only the camera image.
4. **Video.** Screen-record a successful GR00T run the day before as a backup.

Show the dataset too: open one recorded `episode_000000.mp4` to make "demonstration data" concrete.

5.3 Questions you might get

Why not just program the pick-up? You can, and the scripted demo does. But that only works if the ball is exactly on a known spot. A learned policy reads the image, so the same approach scales to tasks nobody can script, like folding a shirt.

How much data? About 60 demonstrations here. Pretraining is what makes that enough.

Why chunks of actions? Smoother motion, fewer model calls, and the model commits to a plan. Re-planning every 8 steps still lets it correct mistakes.

Does it generalise? Within reason: other spots on the plate, small lighting changes. Not to a new task it was never shown.

Why the cloud? Running GR00T needs a GPU with about 16 GB of memory and fine-tuning about 40 GB; the laptop has 8 GB.

Part II
Build and run instructions

6 What you need

Table 2: Parts. Nothing exotic; most of it you already have. Beyond these, the only possible cost is cloud GPU time for GR00T, which has free options (Section 15).

Part	Qty	Notes
SG90 micro servo (with horns and screws)	5	base, shoulder, elbow, wrist, gripper
Arduino Nano + USB cable	1	the cable must carry data, not only charge
Adjustable bench power supply	1	set to 5.5 V, current limit 3 A
Breadboard + male–male jumper wires	–	to share power between servos; with no breadboard, join the 5 red wires together and the 5 brown wires together with spare wire
Superglue (CA)	–	glues each servo horn into its slot
Ping-pong ball (40 mm)	1	or print two `12_ball_half` and glue them
Phone + webcam app	1	Phone Link (Android) or Iriun / DroidCam (any phone)
PLA filament	≈450 g	Bambu printer
Optional: 470–1000 μF capacitor	1	across the servo power rail; smooths current spikes

7 Printing

All STLs are in `cad/stl`. They are generated from `cad/arm.py`; if a fit is off, change the number at the top of that file and re-run it (Section 7.1).

Table 3: Print list. PLA, 0.2 mm layers, 3 walls, 20% infill unless noted. No part needs supports.

File	Qty	How to print
`00_fit_test`	1	**print first**, flat
`01_base`	1	upside down (top plate on the bed)
`02_turntable`	1	disc flat on the bed
`03_upper_arm`, `04_forearm`	1 each	flat
`05_gripper_body`	1	flat, as exported
`06_gripper_jaw`	1	flat, as exported
`07_baseplate`	1	flat; 250×200 mm, the longest print (15% infill is fine)
`08_camera_mast`	1	standing up, 4 walls
`09_phone_cradle`	1	back face down (as exported)
`10_pin`	1	lying flat
`11_cup`	1	upright
`12_ball_half`	2	only if you have no ping-pong ball; open side down

7.1 Fit test first

The fit test has one servo pocket and one horn slot. An SG90 should drop into the pocket with a little play and its tabs should rest flat; a double-arm horn should press into the slot snugly. If the pocket is tight, increase `CLEAR` in `cad/arm.py`; if the horn is loose, decrease `HORN_SLOT_W`. Rebuild with:

```
cd cad
uv run --python 3.12 --with cadquery python arm.py
```

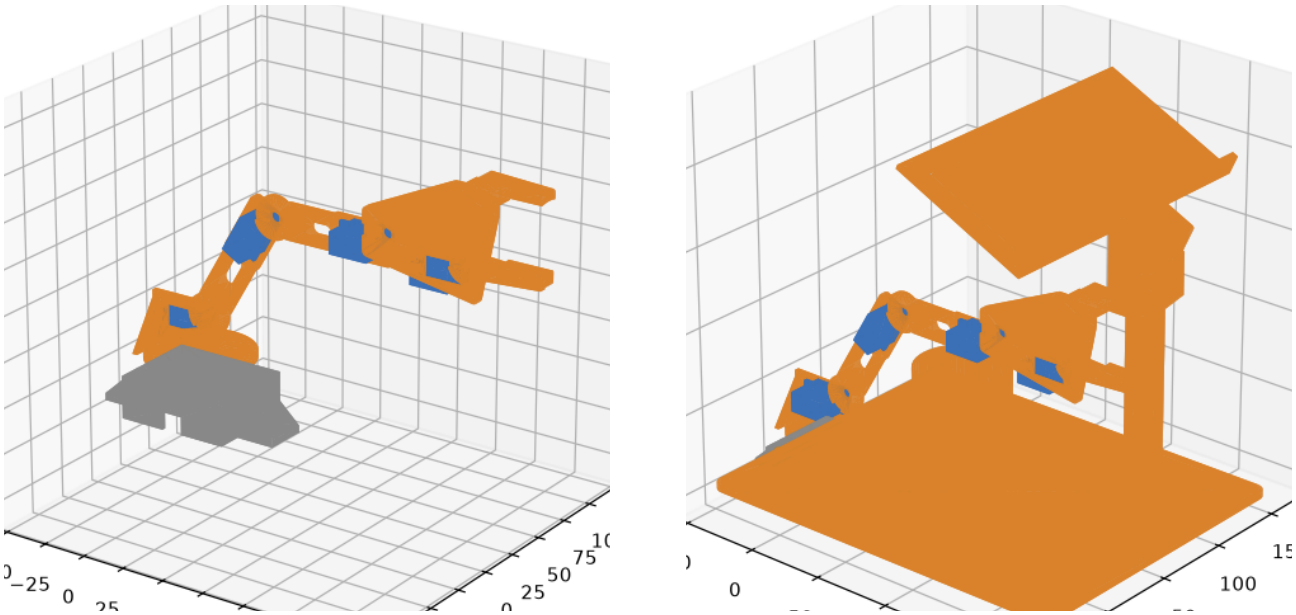


Figure 3: Left: the assembled arm in its home pose (blue = servos). Right: the work cell with the baseplate, cup and phone camera mast. Both were checked for collisions in software (`cad/assembly.py`): every marked ball spot and the cup can be reached without hitting the plate or the camera.

8 Laptop software setup

1. Install **uv** (Python manager) and **Arduino CLI** in PowerShell:

   ```
   powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
   winget install ArduinoSA.CLI
   ```

2. Get the code and install the Python packages:

   ```
   gh repo clone Yishane/gr00t-arm
   cd gr00t-arm\software
   uv sync
   ```

3. Phone as webcam: install **Iriun Webcam** or **DroidCam** on the phone and the PC (or on Android use Windows Phone Link → "Use as connected camera"). It shows up as a normal webcam. The laptop's own camera is usually index 0 and the phone 1 or 2.

9 Wiring

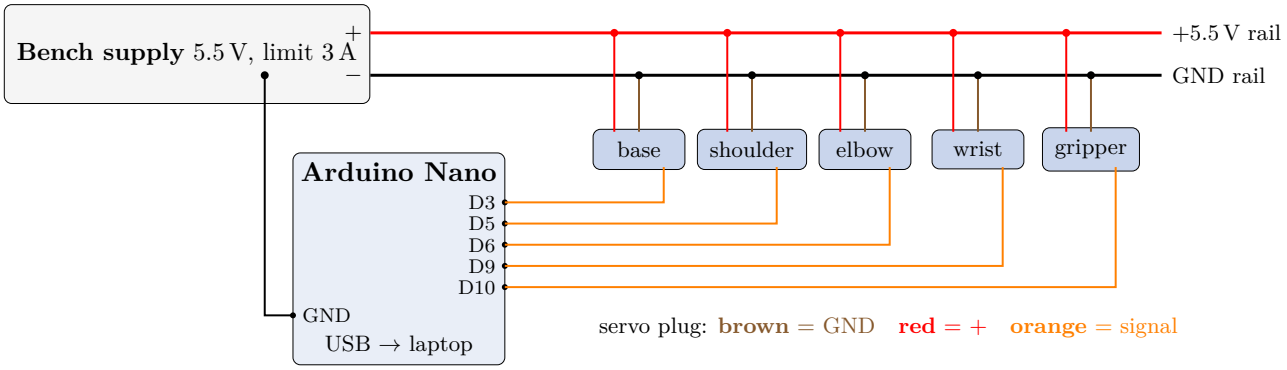


Figure 4: Wiring. Servo power comes from the bench supply, *never* from the Nano's 5V pin. The Nano's GND must connect to the supply's negative so the signals have a common reference. Orange signal wires go to D3, D5, D6, D9, D10 in the order base, shoulder, elbow, wrist, gripper.

1. Set the bench supply to **5.5 V** with the current limit at **3 A**, output off.
2. Bench supply + to the breadboard's red rail, − to the blue rail. (Optional: the capacitor across the two rails, its stripe/negative leg to the blue rail.)
3. Each servo: red to the red rail, brown to the blue rail (use jumpers into the servo's 3-pin plug), orange to its Nano pin from Fig. 4.
4. One jumper from a Nano GND pin to the blue rail.
5. Nano to the laptop by USB.

Why the separate supply

Five SG90s can pull over 2 A when they start moving together. USB gives 0.5 A: the Nano would reset and the arm would twitch. If the supply hits its current limit (the CC light comes on) something is jammed; turn it off and check.

9.1 Upload the firmware

Find the Nano's port in Device Manager → Ports (e.g. COM5), then:

```
cd gr00t-arm\firmware
.\upload.ps1 COM5
```

The script tries the "old bootloader" (most Nano clones) and then the new one. The firmware listens on USB serial for lines like `90,90,90,90,125` (five angles) and moves each servo smoothly toward its target.

10 Assembly

The one rule

Every servo must be at **90 °** when its horn goes on, with the part it drives in the **home pose**: arm facing forward, upper arm straight up, forearm horizontal pointing forward, gripper in line with the forearm, jaw closed (parallel to the fixed finger). The software assumes this; get it right and calibration is only small trims.

1. **Center all servos.** Wire all five servos with nothing attached, power on, and run

   ```
   cd gr00t-arm\software
   uv run calibrate.py
   > center
   ```

   All five turn to 90 °. Leave them powered while fitting horns.

2. **Glue the horns.** Put a drop of CA in each printed part's horn slot (the long groove on the underside) and press a double-arm horn in, hub facing out. Parts with a slot: turntable, upper arm, forearm, gripper body, gripper jaw. Let it set for a few minutes.

3. **Base servo.** Drop the base servo into `01_base` from the top, shaft up, cable out through the notch. Screw the two tabs down with the servo's screws.

4. **Turntable.** Press the turntable's horn onto the base servo spline so the bracket's flat face (where the shoulder servo goes) faces *left* and the arm will swing forward/back. Put the small horn screw through the access hole to lock it.

5. **Shoulder servo.** Slide it into the bracket from the left face, tabs flat on the plate, shaft pointing left, body behind the plate. Screw the tabs.

6. **Upper arm.** Slot side toward the servo; press onto the shoulder spline pointing *straight up*. Horn screw through the access hole.

7. **Elbow servo** into the pocket at the top of the upper arm, from the outer face, shaft pointing left. Screw the tabs.

8. **Forearm** onto the elbow spline, pointing *forward, horizontal*. Horn screw.

9. **Wrist servo** into the forearm's end pocket the same way.

10. **Gripper body** onto the wrist spline, *in line with the forearm*. Horn screw.

11. **Gripper servo** into the gripper body's pocket, shaft pointing the same way.

12. **Jaw** onto the gripper spline with the jaw *closed*: parallel to the fixed finger. Horn screw.

13. **Work cell.** Set the arm base onto the baseplate's two pegs (inside the engraved outline; spots A1–C3 in front, the cup ring to the left). Push the mast into its socket, the pin into the mast top, and the phone cradle onto the pin. Clamp or tape the baseplate's back edge to the desk.

14. **Phone.** Landscape on the cradle, resting on the lip, *rear camera near the lip*. Open the webcam app.

15. **Cables.** Leave slack at every joint so nothing pulls when the arm turns.

11 Calibration

Still in `calibrate.py`:

1. `home` — the arm should go to the home pose.
2. `test` — moves each joint a little and tells you which way it should go. If one goes the wrong way, type `flip <joint>` (0 base, 1 shoulder, 2 elbow, 3 wrist).
3. If a joint is a few degrees off in the home pose: `trim <joint> <degrees>`, e.g. `trim 1 -4`.
4. Gripper: `s 4 <deg>` to try angles. Find an angle that is well open, and one that squeezes the ball firmly without the servo buzzing hard; set them with `grip open 125` and `grip closed 85` (your numbers).
5. `save`, then `quit`. This writes `software/calibration.json`.

12 First moves

```
uv run autodemo.py --spot B2 --camera 1
```

Put the ball on spot B2 first. The arm should pick it up and drop it in the cup. Then try driving it yourself:

```
uv run teleop.py --camera 1
```

Click the window, then: W/S forward/back, A/D left/right, R/F up/down, Q/E tilt the gripper, Space open/close, H home, Esc quit. If it says "can't reach that", the target is outside the arm's reach or would hit the plate; it refuses rather than crashing. Add `-sim` to any script to try it without hardware.

13 Collecting demonstrations

1. **Automatic (recommended):**

   ```
   uv run autodemo.py --record 60 --camera 1
   ```

   It names a random spot; you place the ball and press Enter; it picks and places while recording. 60 demos take about 20 minutes.

2. **By hand (optional, adds variety):** in `teleop.py`, Enter starts a recording, Enter again saves it, Backspace throws it away.

Good data

Keep the camera, lighting and cup exactly where they will be during the demo. Place the ball a little off-center sometimes. Delete any failed demo folder from `data/raw/pick_ball`.

Then export for GR00T:

```
uv run export_dataset.py
```

This writes `data/lerobot/pick_ball` (Table 1). GR00T's own data loader was used to check this format.

14 Training ACT on the laptop (free, optional but recommended)

```
cd software\act
uv sync
uv run export_act.py
uv run train_act.py --steps 20000
uv run ..\run_policy.py act --camera 1 ^
    --checkpoint ..\..\outputs\act\pick_ball\checkpoints\020000\pretrained_model
```

Training runs at about 2.3 steps per second on the RTX 5050 (about 2.5 hours for 20k steps). If ACT picks up the ball, the data is good and GR00T will learn from it too.

15 Fine-tuning and running GR00T in the cloud

15.1 One-time accounts (you do these)

1. **Hugging Face**: make a free account at https://huggingface.co, open https://huggingface.co/nvidia/Cosmos-Reason2-2B and request access (GR00T is built on it). Then create an access token (Settings → Access Tokens, "read").
2. **GPU time.** Fine-tuning needs a GPU with at least 40 GB (L40S, A100 or H100). Try free credits first. **Lightning AI**'s free plan gives monthly credits and a Studio you can SSH into, so the steps below work unchanged. **Modal**'s free plan gives about $30 a month, but it needs a small wrapper script first. Otherwise, rent a machine on a paid cloud such as RunPod or Lambda. Free 16 GB GPUs (Colab, Kaggle) are too small.

This is the only step that can cost money

Everything before this section is free. If you don't want to pay and there are no free credits, stop after ACT: it uses the same data and the same control loop, and it is a fine live demo (Section 5.2, option 2). Check the free plans' current terms on their pricing pages before signing up.

15.2 Steps

1. Start a Linux GPU machine (Ubuntu, L40S/A100/H100, ≥100 GB disk). Note its SSH address and port.
2. Copy the GR00T scripts and the dataset up (from the `gr00t-arm` folder):

   ```
   scp -P <port> -r software\gr00t data\lerobot\pick_ball root@<ip>:~/
   ```

3. Log in and set up (installs GR00T and downloads the base model, ≈20 min):

   ```
   ssh -p <port> root@<ip>
   export HF_TOKEN=hf_your_token
   bash ~/gr00t/setup_cloud.sh
   ```

4. Fine-tune (about 1–2 hours; use `tmux` so it survives a disconnect):

```
tmux
bash ~/gr00t/finetune.sh ~/pick_ball
```

At the end it plots predicted vs. recorded actions for episode 0 in `/tmp/open_loop_eval`. The lines should overlap. Flat predicted lines mean something is wrong with the data.

5. Serve the model:

```
bash ~/gr00t/serve.sh
```

6. On the laptop, open a tunnel in one terminal and run the arm in another:

```
ssh -p <port> -L 5555:localhost:5555 root@<ip>
uv run run_policy.py gr00t --host localhost --camera 1
```

7. **Stop the cloud machine when you're done**: it bills by the hour.

Cost and time

A paid cloud GPU costs roughly $1–3 per hour. Setup plus one fine-tune plus testing is a few hours, so budget $10–20 for the first run, or use free credits if they cover it. Download the checkpoint folder (`~/gr00t_arm_ckpt`) if you want to reuse it later without retraining.

16 Troubleshooting

Symptom	Fix
"No Arduino found"	Check the USB cable carries data; pass `-port COM5`.
Upload fails	Try the other bootloader (the script does both); close other programs using the port.
Servos twitch / Nano resets	Servo power must come from the supply; Nano GND tied to supply −; add the capacitor.
A joint moves the wrong way	`calibrate.py`: `test`, then `flip <joint>`.
Arm sags or buzzes holding pose	Normal for SG90s near their limit; keep the gripper light. MG90S servos are a drop-in upgrade.
"Camera N didn't open"	Try `-camera 0`, `1`, `2`; make sure the phone app is running.
"can't reach that" in teleop	The target is out of reach or the gripper would hit the plate. Move back.
GR00T: `GatedRepoError`	Your HF account hasn't been granted Cosmos-Reason2-2B access yet, or `HF_TOKEN` is wrong.
`run_policy`: server not reachable	Is `serve.sh` running and the SSH tunnel open?
Policy runs but misses	More demos, more varied ball positions, same camera position as in training.

17 Where everything is

`cad/` `arm.py` (all parts, parametric), `assembly.py` (collision and reach checks), `stl/`, `renders/`

`firmware/` `arm/arm.ino`, `upload.ps1`

software/ `config.py` (geometry, calibration), `kinematics.py` (+ tests), `calibrate.py`, `teleop.py`, `autodemo.py`, `recorder.py`, `export_dataset.py`, `policy_client.py`, `run_policy.py`

software/gr00t/ `gr00t_arm_config.py`, `setup_cloud.sh`, `finetune.sh`, `serve.sh`

software/act/ `export_act.py`, `train_act.py`

context.md running notes on the project's state

References

[1] NVIDIA et al., "GR00T N1: An Open Foundation Model for Generalist Humanoid Robots," arXiv:2503.14734, 2025.

[2] NVIDIA, "Isaac GR00T" (code, GR00T N1.7 model, fine-tuning guides), https://github.com/NVIDIA/Isaac-GR00T.

[3] T. Z. Zhao, V. Kumar, S. Levine, C. Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware," RSS 2023, arXiv:2304.13705.

[4] Hugging Face, "LeRobot," https://github.com/huggingface/lerobot.